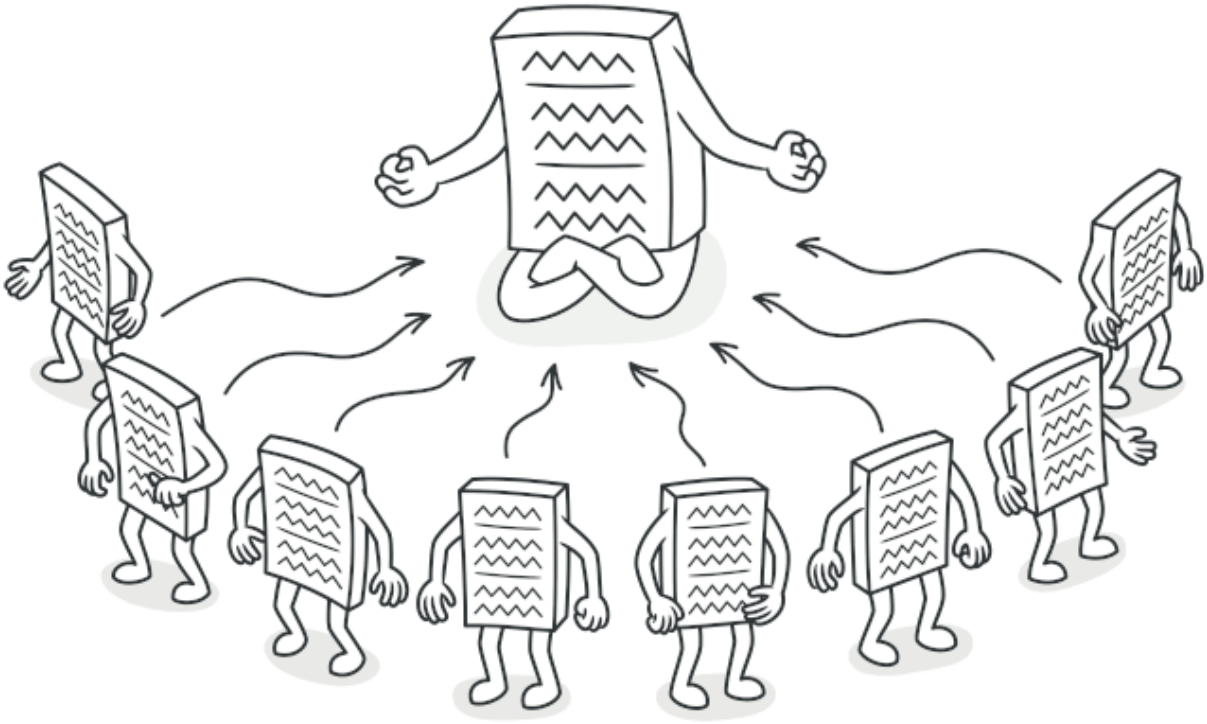
Singleton

Absicht

Singleton ist ein Erstellungsentwurfsmuster, mit dem Sie sicherstellen können, dass eine Klasse nur eine Instanz hat, während Sie gleichzeitig einen globalen Zugriffspunkt auf diese Instanz bereitstellen.



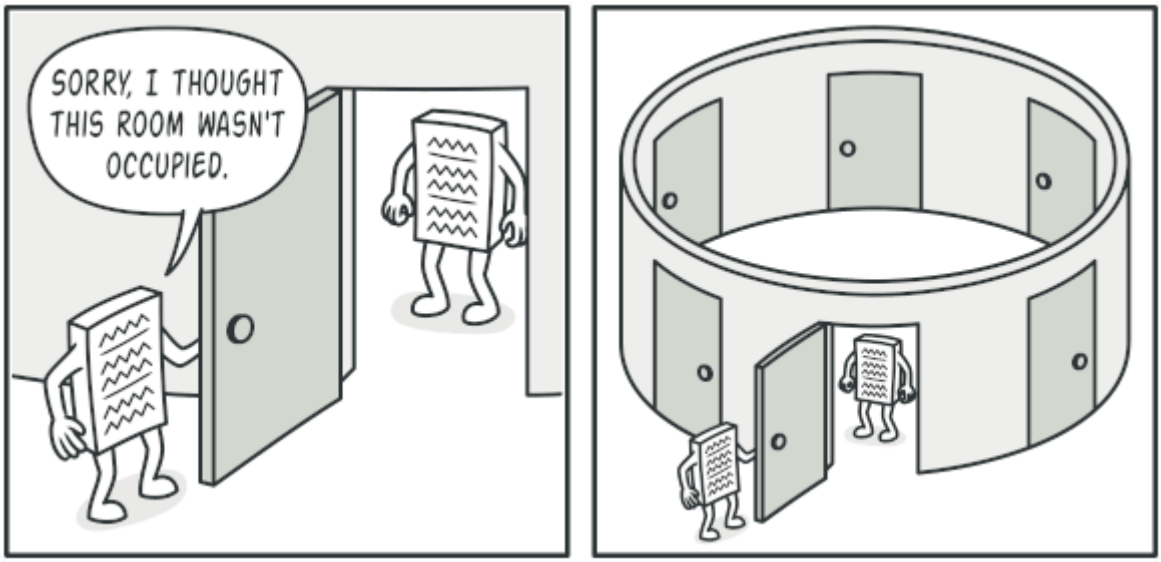
Problem

Das Singleton-Muster löst zwei Probleme gleichzeitig und verstößt damit gegen das *Single Responsibility Principle* :

1. **Stellen Sie sicher, dass eine Klasse nur eine einzige Instanz hat** . Warum sollte jemand die Anzahl der Instanzen einer Klasse kontrollieren wollen? Der häufigste Grund hierfür ist die Kontrolle des Zugriffs auf gemeinsam genutzte Ressourcen, beispielsweise eine Datenbank oder eine Datei.

   So funktioniert es: Stellen Sie sich vor, Sie haben ein Objekt erstellt, möchten aber nach einer Weile ein neues erstellen. Statt eines neuen Objekts erhalten Sie das bereits erstellte.

   Beachten Sie, dass dieses Verhalten mit einem regulären Konstruktor nicht implementiert werden kann, da ein Konstruktoraufruf konstruktionsbedingt immer ein neues Objekt zurückgeben **muss .**



Den Kunden ist möglicherweise nicht einmal bewusst, dass sie die ganze Zeit mit demselben Objekt arbeiten.

2. **Stellen Sie einen globalen Zugriffspunkt für diese Instanz bereit** . Erinnern Sie sich an die globalen Variablen, die Sie (also ich) zum Speichern wichtiger Objekte verwendet haben? Sie sind zwar sehr praktisch, aber auch sehr unsicher, da jeder Code den Inhalt dieser Variablen überschreiben und die App zum Absturz bringen kann.

   Wie bei einer globalen Variable ermöglicht das Singleton-Muster den Zugriff auf ein Objekt von überall im Programm. Es schützt die Instanz jedoch auch vor dem Überschreiben durch anderen Code.

   Dieses Problem hat aber noch eine andere Seite: Sie möchten nicht, dass der Code, der Problem Nr. 1 löst, über das gesamte Programm verstreut ist. Es ist viel besser, ihn in einer Klasse zu haben, insbesondere wenn der restliche Code bereits davon abhängt.

Heutzutage ist das Singleton-Muster so populär geworden, dass man etwas als Singleton bezeichnet, selbst wenn es nur eines der aufgeführten Probleme löst.

Lösung

Allen Implementierungen des Singletons sind diese beiden Schritte gemeinsam:

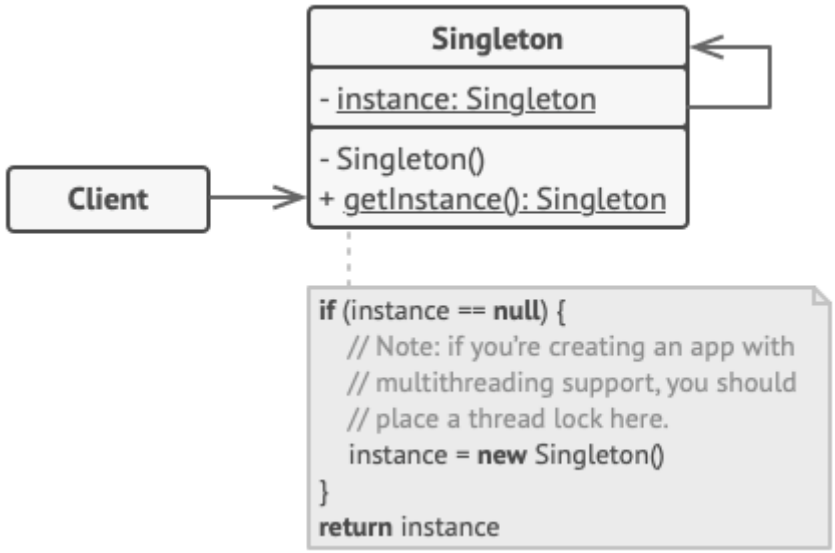
- Machen Sie den Standardkonstruktor privat, um zu verhindern, dass andere Objekte den `new`Operator mit der Singleton-Klasse verwenden.
- Erstellen Sie eine statische Erstellungsmethode, die als Konstruktor fungiert. Im Hintergrund ruft diese Methode den privaten Konstruktor auf, um ein Objekt zu erstellen und es in einem statischen Feld zu speichern. Alle folgenden Aufrufe dieser Methode geben das zwischengespeicherte Objekt zurück.

Wenn Ihr Code Zugriff auf die Singleton-Klasse hat, kann er die statische Methode des Singletons aufrufen. Bei jedem Aufruf dieser Methode wird also immer dasselbe Objekt zurückgegeben.

Analogie zur realen Welt

Die Regierung ist ein hervorragendes Beispiel für das Singleton-Muster. Ein Land kann nur eine offizielle Regierung haben. Unabhängig von der persönlichen Identität der Regierungsmitglieder ist der Titel „Regierung von X“ ein globaler Zugangspunkt, der die verantwortliche Personengruppe identifiziert.

Struktur



1. Die **Singleton** -Klasse deklariert die statische Methode `getInstance`, die dieselbe Instanz ihrer eigenen Klasse zurückgibt.

   Der Konstruktor des Singletons sollte vor dem Clientcode verborgen sein. Der Aufruf der `getInstance`Methode sollte die einzige Möglichkeit sein, das Singleton-Objekt abzurufen.

Pseudocode

In diesem Beispiel fungiert die Datenbankverbindungsklasse als **Singleton** . Diese Klasse verfügt über keinen öffentlichen Konstruktor, daher ist der einzige Weg, ihr Objekt abzurufen, der Aufruf der `getInstance`Methode. Diese Methode speichert das zuerst erstellte Objekt zwischen und gibt es bei allen nachfolgenden Aufrufen zurück.

```
// The Database class defines the `getInstance` method that lets
// clients access the same instance of a database connection
// throughout the program.
class Database is
    // The field for storing the singleton instance should be
    // declared static.
    private static field instance: Database

    // The singleton's constructor should always be private to
    // prevent direct construction calls with the `new`
    // operator.
    private constructor Database() is
        // Some initialization code, such as the actual
        // connection to a database server.
        // ...

    // The static method that controls access to the singleton
    // instance.
    public static method getInstance() is
        if (Database.instance == null) then
            acquireThreadLock() and then
                // Ensure that the instance hasn't yet been
                // initialized by another thread while this one
                // has been waiting for the lock's release.
                if (Database.instance == null) then
                    Database.instance = new Database()
        return Database.instance

    // Finally, any singleton should define some business logic
    // which can be executed on its instance.
    public method query(sql) is
        // For instance, all database queries of an app go
        // through this method. Therefore, you can place
        // throttling or caching logic here.
        // ...

class Application is
    method main() is
        Database foo = Database.getInstance()
        foo.query("SELECT ...")
        // ...
        Database bar = Database.getInstance()
        bar.query("SELECT ...")
        // The variable `bar` will contain the same object as
        // the variable `foo`.
```

Anwendbarkeit

Verwenden Sie das Singleton-Muster, wenn eine Klasse in Ihrem Programm nur eine einzige Instanz haben soll, die allen Clients zur Verfügung steht. Beispielsweise ein einzelnes Datenbankobjekt, das von verschiedenen Teilen des Programms gemeinsam genutzt wird.
Das Singleton-Muster deaktiviert alle anderen Möglichkeiten zum Erstellen von Objekten einer Klasse mit Ausnahme der speziellen Erstellungsmethode. Diese Methode erstellt entweder ein neues Objekt oder gibt ein vorhandenes zurück, falls es bereits erstellt wurde.

Verwenden Sie das Singleton-Muster, wenn Sie eine strengere Kontrolle über globale Variablen benötigen.
Im Gegensatz zu globalen Variablen garantiert das Singleton-Muster, dass es nur eine Instanz einer Klasse gibt. Außer der Singleton-Klasse selbst kann nichts die zwischengespeicherte Instanz ersetzen.

Beachten Sie, dass Sie diese Einschränkung jederzeit anpassen und die Erstellung einer beliebigen Anzahl von Singleton-Instanzen zulassen können. Der einzige Codeteil, der geändert werden muss, ist der Hauptteil der `getInstance`Methode.

So implementieren Sie

1. Fügen Sie der Klasse ein privates statisches Feld zum Speichern der Singleton-Instanz hinzu.
2. Deklarieren Sie eine öffentliche statische Erstellungsmethode zum Abrufen der Singleton-Instanz.
3. Implementieren Sie eine verzögerte Initialisierung innerhalb der statischen Methode. Diese sollte beim ersten Aufruf ein neues Objekt erstellen und in das statische Feld einfügen. Bei allen nachfolgenden Aufrufen sollte die Methode stets diese Instanz zurückgeben.
4. Machen Sie den Konstruktor der Klasse privat. Die statische Methode der Klasse kann den Konstruktor weiterhin aufrufen, die anderen Objekte jedoch nicht.
5. Gehen Sie den Clientcode durch und ersetzen Sie alle direkten Aufrufe des Konstruktors des Singletons durch Aufrufe seiner statischen Erstellungsmethode.

Für und Wider

- Sie können sicher sein, dass eine Klasse nur eine einzige Instanz hat.
- Sie erhalten einen globalen Zugriffspunkt zu dieser Instanz.
- Das Singleton-Objekt wird nur initialisiert, wenn es zum ersten Mal angefordert wird.

- Verstößt gegen das *Single Responsibility Principle* . Das Muster löst zwei Probleme gleichzeitig.
- Das Singleton-Muster kann ein schlechtes Design verschleiern, beispielsweise wenn die Komponenten des Programms zu viel voneinander wissen.
- Das Muster erfordert eine spezielle Behandlung in einer Multithread-Umgebung, sodass mehrere Threads ein Singleton-Objekt nicht mehrmals erstellen.

- Es kann schwierig sein, den Client-Code des Singletons einem Unit-Test zu unterziehen, da viele Testframeworks bei der Erstellung von Mock-Objekten auf Vererbung setzen. Da der Konstruktor der Singleton-Klasse privat ist und das Überschreiben statischer Methoden in den meisten Sprachen nicht möglich ist, müssen Sie sich eine kreative Möglichkeit zum Mocking des Singletons überlegen. Oder schreiben Sie die Tests einfach nicht. Oder verwenden Sie das Singleton-Muster nicht.

Beziehungen zu anderen Mustern

- Eine **Facade** -Klasse kann oft in ein **Singleton** umgewandelt werden , da in den meisten Fällen ein einzelnes Fassadenobjekt ausreicht.
- **Flyweight würde Singleton** ähneln, wenn es gelänge, alle gemeinsamen Zustände der Objekte auf ein einziges Flyweight-Objekt zu reduzieren. Es gibt jedoch zwei grundlegende Unterschiede zwischen diesen Mustern:
    1. Es sollte nur eine Singleton-Instanz geben, während eine *Flyweight*- Klasse mehrere Instanzen mit unterschiedlichen intrinsischen Zuständen haben kann.
    2. Das *Singleton* -Objekt kann veränderlich sein. Flyweight-Objekte sind unveränderlich.
- **Abstract Factory** , **Builder** und **Prototypen** können alle als **Singletons** implementiert werden .